

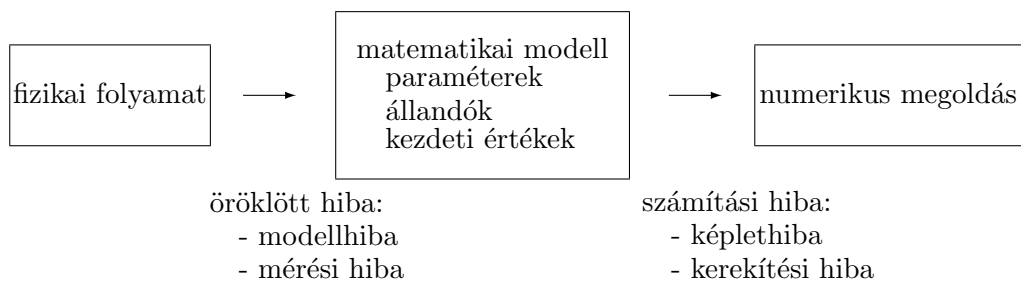
1. fejezet

Bevezetés

Ebben a fejezetben először a numerikus analízis feladatát ismertetjük, majd alapfogalmakat vizsgálunk. A matematikai számítások közben felmerülő hibák több fajtáját definiáljuk, bevezetjük egy matematikai feladat illetve egy numerikus algoritmus stabilitásának, műveletigényének, tárolási igényének fogalmát. Ezután az egész és valós számok számítógépen történő tárolásának különféle szabványait ismertetjük, és a véges sok számjegyen történő tárolás miatt fellépő problémákat vizsgáljuk.

1.1. A numerikus analízis feladata, alapfogalmak

A fizikai valóság folyamatainak leírására és a vizsgált fizikai változók jelen ill. jövőbeli értékének meghatározására szolgáló matematikai számítások vázlatos menetét az 1.1. ábrával lehet szemléltetni.



1.1. ábra.

Az első lépés a vizsgált folyamat leírása, matematikai modellezése. Ez a lépés a megfelelő tudományág (fizika, kémia, biológia, közgazdaságtan stb.) feladata. A kapott modellben sokszor szerepelnek paraméterek, állandók, kezdeti feltételek, amelyeket általában megfigyeléssel, méréssel állapíthatunk meg. Ha a modell és az abban szereplő paraméterek ismertek, akkor használhatjuk a matematikai modellt a fizikai rendszerre vonatkozó kérdések megválaszolására. A fizikai rendszerre, ill. az azt leíró matematikai modellre feltehetünk kvalitatív kérdéseket (pl. létezik-e egyértelmű megoldása a matematikai feladatnak, van-e határértéke a vizsgált változónak, periodikus-e a folyamat stb.) vagy kvantitatív kérdéseket (mi a vizsgált fizikai változó értéke egy adott időpontban, mi a pontos vagy közelítő megoldása a matematikai modellnek stb.). A kvalitatív kérdésekre az adott modellhez tartozó matematikai szakterület keresi a választ, a kvantitatív kérdésekkel pedig a numerikus analízis foglalkozik. A numerikus analízis feladata matematikai feladatok numerikus eredményének aritmetikai műveletekkel (osztás, szorzás, összeadás, kivonás) való pontos vagy közelítő megoldása.

Az 1.1. ábrán leírt folyamattal számolt fizikai változó értéke általában nem egyezik meg pontosan a változó tényleges értékével. Az elkövetett hibát két nagyobb kategóriára bontjuk:

öröklött hiba és számítási hiba. Az öröklött hiba az első lépésben, azaz a fizikai folyamat matematikai modellel való helyettesítésekor elkövetett hiba. Ezt is két részre oszthatjuk: *modellhiba* és *mérési hiba*. A modellhiba abból adódik, hogy a matematikai modellek levezetésére használt törvények idealizáltak, csak „közelítései” a valóságnak. A mérési hiba az a hiba, amit azért kapunk, hogy a matematikai modellben a valódi paramétereknek, kezdeti feltételeknek csak mért, így közelítő értékét használjuk a tényleges értékek helyett.

A számítási hibát is két részre bontjuk, *képlethibára* és *kerekítési hibára*. A képlethiba az a hiba, amit akkor követünk el, amikor egy matematikai kifejezés pontos értéke helyett annak közelítő értékét használjuk.

1.1. példa. Tegyük fel, hogy az $f(x) = \sin x$ függvény értékét kell kiszámítanunk egy megadott x pontban. Az $f(x)$ függvényérték helyett kiszámíthatjuk pl. az f függvény ötödrendű Taylor-polinomját: $T_5(x) = x - x^3/3! + x^5/5!$. A Taylor-tétel (2.5. tétel) szerint ha $f(x)$ -et $T_5(x)$ -szel helyettesítjük, akkor a közelítés hibája $\frac{f^{(6)}(\xi)}{6!}x^6 = -\frac{\sin \xi}{6!}x^6$ alakban írható fel. Ez a hiba (azaz a módszerünk képlethibája) kicsi, ha x közel van 0-hoz. $\square$

A kerekítési hiba abból adódik, hogy a számítógépen egy valós számot csak véges sok tizedesjegy pontossággal tudunk tárolni, így általában már a valós számok tárolásakor követünk el hibát. Kerekítési hiba lép fel az aritmetikai műveletek végzése közben is: a számítógép az egyes aritmetikai műveletek eredményeit adott számú tizedesjegyre kerekítés után tárolja/használja tovább. A kerekítési hibával bővebben az 1.2.–1.4. szakaszokban foglalkozunk.

Egy numerikus módszer levezetésekor az első kérdés amit vizsgálunk kell, a módszer képlethibája, hiszen egy numerikus közelítő érték csak akkor hasznos, ha azt is tudjuk róla, hogy mekkora hibával közelíti a pontos értéket. A következő fogalom, ami egy numerikus módszerrel kapcsolatban felmerül, a *stabilitás*. Ezt a fogalmat kétféle értelemben is használjuk. Beszélhetünk egy matematikai modell vagy feladat stabilitásáról, vagy egy numerikus módszer stabilitásáról. Kezdjük egy példával.

1.2. példa. Tekintsük a

$$8x + 917y = 1794$$

$$7x + 802y = 1569.$$

lineáris egyenletrendszer. Ennek megoldása $x = -5$ és $y = 2$. Ha viszont a második egyenletben x együtthatóját 7.01-re¹ változtatjuk, akkor a

$$8x + 917y = 1794$$

$$7.01x + 802y = 1569$$

egyenletrendszer megoldása $x = -1.232562589$ és $y = 1.967132499$ lesz (9 tizedesjegy pontossággal). Azt tapasztaljuk, hogy 0.14%-os változás az együtthatóban a megoldás 75.3%-os ill. 1.6%-os változását eredményezte. $\square$

Azt mondjuk, hogy egy matematikai feladat *korrekt* vagy *stabil*, ha „kis” változás a feladat paramétereiben a megoldás „kis” változását idézi csak elő. Ellenkező esetben *inkorrekt* vagy *instabil feladatról* beszélünk. Az előző példában vizsgált egyenletrendszer tehát e szerint a terminológia szerint egy inkorrekt feladat.

¹Ebben a jegyzetben, hogy a számítógépek és programozási nyelvek által használt jelöléssel összhangban legyen, a törtszámoknál tizedespontot használunk tizedesvessző helyett.

Egy numerikus algoritmust a kerekítési hibákra nézve *stabilnak* nevezzük, ha a kerekítési hibák nem befolyásolják jelentősen a számított végeredményt. Ha a kerekítési hibák miatt a számított végeredmény jelentősen eltér a számítandó értéktől, akkor az algoritmust *instabilnak* nevezzük. A következő példában egy instabil algoritmust mutatunk be.

1.3. példa. Tekintsük a következő három, rekurzív definícióval megadott sorozatot:

$$\begin{aligned} x_n &= \frac{1}{3}x_{n-1}, & x_0 &= 1, \\ y_n &= 2y_{n-1} - \frac{5}{9}y_{n-2}, & y_0 &= 1, \quad y_1 = \frac{1}{3}, \\ z_n &= \frac{13}{3}z_{n-1} - \frac{4}{3}z_{n-2}, & z_0 &= 1, \quad z_1 = \frac{1}{3}. \end{aligned} \quad (1.1)$$

Könnyen látható, hogy mindhárom képlet az $x_n = y_n = z_n = \frac{1}{3^n}$ számsorozatot definiálja, azaz a három sorozat algebrailag ekvivalens. A gyakorlatban viszont észrevehető különbség van a három képlet között. Az 1.1. táblázatban kinyomtattuk az (1.1) képlettel számított sorozatok első 18 tagjait. A számításokat egyszeres pontossággal végeztük csak, hogy a kerekítési hibák hatását jobban láthassuk. Azt tapasztaljuk, hogy x_n tényleg az $1/3^n$ értékeket állítja elő, viszont az y_n és z_n számolt értékeiben kerekítési hibából származó eltérést láthatunk. Mindkét sorozatnál fellép a hiba, de a z_n esetében a hiba igen gyorsan nő, a 18. tagnál már 100-as nagyságrendű. Azt tapasztaltuk tehát, hogy az x_n képlete egy stabil, a z_n képlete pedig egy instabil módszer $1/3^n$ generálására.

Arról, hogy az előbb vizsgált hibák tényleg a kerekítési hibák rovására írhatók, meggyőződhetünk úgy, hogy megismételjük a számításokat, de most dupla pontosságot használva a számok tárolásához. Az egész listát nem, csak a 18. tag hibáját közöljük: $|y_{18} - 1/3^{18}| = -2.5104e - 13$ és $|z_{18} - 1/3^{18}| = 2.3804e - 07$. Látható, hogy ebben az esetben a számolás hibája sokkal kisebb. $\square$

1.1. táblázat.

n	x_n	y_n	$ y_n - 1/3^n $	z_n	$ z_n - 1/3^n $
2	0.111111	0.111111	2.2352e-08	0.111111	4.4703e-08
3	0.037037	0.037037	4.0978e-08	0.037037	1.8254e-07
4	0.012346	0.012346	6.9849e-08	0.012346	7.3109e-07
5	0.004115	0.004115	1.1688e-07	0.004118	2.9248e-06
6	0.001372	0.001372	1.9465e-07	0.001383	1.1699e-05
7	0.000457	0.000458	3.2442e-07	0.000504	4.6795e-05
8	0.000152	0.000153	5.4071e-07	0.000340	1.8718e-04
9	0.000051	0.000052	9.0117e-07	0.000800	7.4872e-04
10	0.000017	0.000018	1.5019e-06	0.003012	2.9949e-03
11	0.000006	0.000008	2.5032e-06	0.011985	1.1980e-02
12	0.000002	0.000006	4.1721e-06	0.047920	4.7918e-02
13	0.000001	0.000008	6.9535e-06	0.191674	1.9167e-01
14	0.000000	0.000012	1.1589e-05	0.766693	7.6669e-01
15	0.000000	0.000019	1.9315e-05	3.066773	3.0668e+00
16	0.000000	0.000032	3.2192e-05	12.267091	1.2267e+01
17	0.000000	0.000054	5.3653e-05	49.068363	4.9068e+01
18	0.000000	0.000089	8.9422e-05	196.273453	1.9627e+02

A következő fogalom, amit egy (véges sok lépésből álló) numerikus módszernél vizsgálni szoktunk, az algoritmus *műveletigénye* vagy *műveletszáma*. Tekintsünk először egy példát:

1.4. példa. Számítsuk ki a $p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10$ negyedfokú polinom értékét egy megadott x pontban! Természetesen ezt könnyen megtehetjük p képletét és aritmetikai műveleteket használva. A képletben 4 összeadás/kivonás, 4 szorzás és 3 hatványozás szerepel. A hatványozások tulajdonképpen $3+2+1=6$ szorzást jelentenek, azaz összesen 10 szorzásra van szükség a képlet alkalmazásához. Megtehetjük viszont, hogy átalakítjuk p képletét:

$$p(x) = 5x^4 - 8x^3 + 2x^2 + 4x - 10 = (((5x - 8)x + 2)x + 4)x - 10.$$

A p polinomnak ezt az alakját használva már csak 4 összeadás ill. kivonás valamint 4 szorzás kell a képlet kiértékeléséhez. $\square$

Az előző példában bemutatott eljárást megismételhetjük általános n -edfokú polinomokra:

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = ((\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots)x + a_1)x + a_0$$

Ebben a képletben összesen csak n összeadás/kivonás és n szorzás szerepel. Ezt a polinomok kiértékelésére vonatkozó módszert *Horner-eljárásnak* nevezzük. A módszert az 1.5. algoritmussal írhatjuk le.

1.5. algoritmus. Horner-eljárás

INPUT: n - a polinom fokszáma
 $a_n, a_{n-1}, \dots, a_0$ - a polinom együtthatói
 x - ahol a polinomot kiértékeljük
 OUTPUT: p - a polinom értéke az x pontban

```

 $p \leftarrow a_n$ 
for  $i = n - 1, \dots, 0$  do
     $p \leftarrow a_i + px$ 
end do
output( $p$ )
  
```

A számítógépeken egy szorzás ill. osztás elvégzése jelentősen tovább tart, mint egy összeadás vagy kivonás. Ezért egy algoritmus műveletigényén általában a benne szereplő osztások/szorzások számát szokás érteni.

Egy algoritmusra jellemző tulajdonság még az *adattárolási igénye*. Egy 10×10 -es lineáris egyenletrendszer megoldására használt algoritmus esetében az adatok tárolása nem jelenthet problémát, de ugyanez 10000×10000 -es rendszerre már gond lehet. Ilyen mennyiségű adat kezelésekor előnyben részesítünk olyan algoritmusokat, amelyeknek minél kisebb az adattárolási igénye. Például, ha tudjuk, hogy az együtthatómátrixban csak a főátlóban és az alatta ill. felette levő néhány átlóban vannak csak nem nulla elemek (ún. szalagmátrix), akkor mindenképpen célszerű olyan algoritmust használni, amely kihasználja az adatok speciális szerkezetét, és nem tárolja számolás közben a felesleges nullákat. Ilyen módszerre látunk majd példát a 3.5. szakaszban.

1.2. Egész és valós számok tárolása

Legyen I egy b -alapú számrendszerben felírt m jegyű pozitív egész szám:

$$I = (a_{m-1}a_{m-2} \dots a_1a_0)_b, \quad \text{ahol } a_i \in \{0, 1, \dots, b-1\}.$$

Ennek értéke:

$$I = a_{m-1}b^{m-1} + a_{m-2}b^{m-2} + \dots + a_1b + a_0.$$

m jegyen tárolható legnagyobb egész szám tehát az az $I_{\max}$ szám, amelynek minden számjegye $b-1$. Ennek értéke

$$I_{\max} = (b-1)(b^{m-1} + b^{m-2} + \dots + b + 1) = b^m - 1.$$

m jegyen tehát a 0-tól $b^m - 1$ -ig terjedő (b^m db) nemnegatív számokat tudjuk ábrázolni (tárolni). A számítógépen a kettes (bináris) számrendszerben tároljuk az egész számokat. m biten tehát 2^m db számot tudunk ábrázolni. Negatív egész számok tárolására két módszert ismertetünk. Az első az ún. *direkt* vagy *egyenest kód*. Ebben a kódolásban egy bitet lefoglalunk az előjelnek, (ezt hívjuk előjelbitnek), és a maradék $m - 1$ biten tudjuk a szám abszolút értékét tárolni. Ekkor $I_{\max} = 2^{m-1} - 1$, és a legkisebb tárolható egész szám $I_{\min} = -I_{\max}$. Ebben a kódolásban a 0-t kétféleképpen tárolhatjuk: a csupa 0, ill. az $100 \dots 0$ bitsorozattal.

1.6. példa. Az 1.2. táblázatban az $m = 3$ biten, direkt kóddal ábrázolható számokat soroltuk fel. $\square$

1.2. táblázat. Egyenes kód, $m = 3$

I	a tárolt bináris kód
0	000
1	001
2	010
3	011
0	100
-1	101
-2	110
-3	111

A gyakorlatban általában az ún. *kettes komplementes kódot* szokták használni negatív számok tárolására. Legyen I egy egész szám, amit m biten szeretnénk tárolni. Az I helyett a C szám bináris alakját tároljuk, ahol

$$C = \begin{cases} I, & \text{ha } 0 \leq I \leq 2^{m-1} - 1, \\ 2^m + I, & \text{ha } -2^{m-1} \leq I < 0. \end{cases}$$

Ennél a tárolásnál tehát a legnagyobb és a legkisebb ábrázolható szám $I_{\max} = 2^{m-1} - 1$ ill. $I_{\min} = -2^{m-1}$. A feltételek szerint ha $0 \leq I \leq 2^{m-1} - 1$, akkor $C < 2^{m-1}$, azaz C első bitje 0. Ha viszont $-2^{m-1} \leq I < 0$, akkor könnyen ellenőrizhető, hogy $2^{m-1} \leq C \leq 2^m - 1$, azaz C első bitje 1.

A kettes komplementes kód egyik fontos előnye, hogy segítségével a kivonás visszavezethető összeadásra. (Lásd a 4. feladatot!)

1.7. példa. Az 1.3. táblázat az $m = 3$ biten, kettes komplementes kóddal ábrázolható számokat tartalmazza. $\square$

1.3. táblázat. Kettes komplementes kód, $m = 3$

I (decimálisan)	I (binárisan)	C , a tárolt bináris kód
0	000	000
1	001	001
2	010	010
3	011	011
-1	-001	111
-2	-010	110
-3	-011	101
-4	-100	100

A továbbiakban a valós számok tárolását vizsgáljuk. Emlékeztetünk arra, hogy a b alapú számrendszerben felírt

$$x = (x_{m-1}x_{m-2} \dots x_0.x_{-1}x_{-2} \dots)_b, \quad x_i \in \{0, 1, \dots, b-1\},$$

valós szám értéke

$$x = x_{m-1}b^{m-1} + x_{m-2}b^{m-2} + \dots + x_1b + x_0 + \frac{x_{-1}}{b} + \frac{x_{-2}}{b^2} + \dots = \sum_{i=-\infty}^{m-1} x_i b^i.$$

Tekintsük a 126.42 valós számot. Ennek normál alakján vagy az $1.2642 \cdot 10^2$ vagy pedig a $0.12642 \cdot 10^3$ alakot szokás érteni. Mi ebben a jegyzetben az első alakot fogjuk használni. Ennek megfelelően egy $x \neq 0$ valós szám b alapra vonatkozó *normál alakján* az $x = \pm m \cdot b^k$ alakját hívjuk, ahol $1 \leq m < b$. m -et a szám *mantisszájának*, k -t pedig kitevőjének nevezzük. Valós számok, más szóval *lebegőpontos számok* tárolásához a számot felírjuk (valamely b alapot használva) normál alakban, és az előjeles mantisszát, valamint a kitevőt tároljuk. Különböző számítógépek eltérő alapot és bithosszúságot használnak egy valós szám tárolására. Mi most egy IEEE szabványt² ismertetünk valós számok 32 biten (ún. *egyszeres pontosságú*), ill. 64 biten történő (ún. *dupla pontosságú*) tárolására bináris alapot használva. Ezt a kódolást használják az IBM-kompatibilis személyi számítógépek is. Vegyük a szám $x = (-1)^s m \cdot 2^k$ bináris normál alakját, ahol $s \in \{0, 1\}$ és $m = 1.m_1m_2m_3\dots$. Az s értékét az 1. biten tároljuk. A k kitevő helyett annak eltolt értékét, az $e = k + 127$ nemnegatív számot a 2.–9. biteken tároljuk, a mantissza törtrészének első 23 bitre kerekített értékét pedig a 10.–32. biten tároljuk. (A nemnulla szám mantisszájának egész része bináris normál alakban mindig 1-gyel egyenlő, ezt az 1-est nem tároljuk!) A fent említett IEEE szabvány külön definiálja a 0 tárolását, és bevezet két speciális szimbólumot is, az **Inf** (infinity, azaz végtelen) és **NaN** (not-a-number, azaz nem szám) szimbólumokat:

tárolandó szám	s	e (2.–9. bitek)	mantissza bitek (10.–32. bitek)
+0	0	00000000	minden mantissza bit=0
−0	1	00000000	minden mantissza bit=0
+Inf	0	11111111	legalább az egyik mantissza bit=0
−Inf	1	11111111	legalább az egyik mantissza bit=0
+NaN	0	11111111	minden mantissza bit=1
+NaN	1	11111111	minden mantissza bit=1

Az **Inf** szimbólumot a programok használhatják olyan matematikai művelet eredményének tárolására, amelynek értéke végtelen, a **NaN** szimbólumot pedig olyan művelet „eredményének” tárolására, amely nem definiált (pl. nullával való osztás eredménye vagy negatív szám négyzetgyöke valós számok körében). Mindkét szimbólumnak lehet pozitív vagy negatív előjele. A szabvány definíciójából következik, hogy az $e = (11111111)_2 = 255$ azaz a $k = 128$ kitevő az **Inf** és **NaN** speciális szimbólumoknak van fenntartva. A véges valós számok esetén $0 \leq e \leq 254$, így k lehetséges értékei $-127 \leq k \leq 127$. A legkisebb pozitív valós szám tehát a $k = -127$ kitevőhöz és az $(1.00\dots 01)_2$ mantisszához tartozik. Ennek értéke $(1 + 1/2^{23})2^{-127} \approx 10^{-38}$. A legnagyobb (véges) valós szám pedig $x_{\max} = (1.11\dots 1)_2 2^{127} = (2 - 2^{-23})2^{127} \approx 10^{38}$.

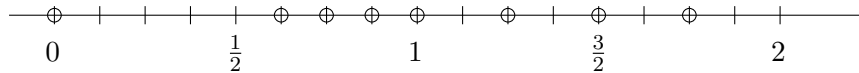
64 biten történő tárolás az előzőekhez hasonlóan történik: az $e = k + 1023$ eltolt kitevőt a 2.–12. biten, a mantissza törtrészét pedig a 13.–64. biten tároljuk. Ekkor a tárolható pozitív számok tartománya körülbelül $10^{-308} - 10^{308}$ lesz.

1.8. példa. Tegyük fel, hogy 4 biten szeretnénk valós számokat tárolni, bináris normál alak segítségével. Ezt megtehetjük pl. úgy, hogy az 1. biten a szám előjelét, a 2. biten a bináris normál alak eltolt kitevőjét, $e = k + 1$ -et, a 3.–4. biten pedig a mantissza tört részének első két bitjét tároljuk. (Az **Inf** és **NaN** szimbólumokat nem definiáljuk most.) A fenti szabály szerint négy biten ábrázolható nemnegatív valós számokat az 1.4. táblázat ill. az 1.2. ábra tartalmazza. $\square$

²IEEE Binary Floating Point Arithmetic Standard, 754-1985.

1.4. táblázat. Nemnegatív valós számok 4 biten

s	e	m	x
0	0	00	0
0	0	01	$(1.01)_2 \cdot 2^{-1} = (1 + \frac{1}{4})\frac{1}{2} = \frac{5}{8}$
0	0	10	$(1.10)_2 \cdot 2^{-1} = (1 + \frac{1}{2})\frac{1}{2} = \frac{3}{4} = \frac{6}{8}$
0	0	11	$(1.11)_2 \cdot 2^{-1} = (1 + \frac{1}{2} + \frac{1}{4})\frac{1}{2} = \frac{7}{8}$
0	1	00	$(1.00)_2 \cdot 2^0 = 1 = \frac{8}{8}$
0	1	01	$(1.01)_2 \cdot 2^0 = 1 + \frac{1}{4} = \frac{10}{8}$
0	1	10	$(1.10)_2 \cdot 2^0 = 1 + \frac{1}{2} = \frac{12}{8}$
0	1	11	$(1.11)_2 \cdot 2^0 = 1 + \frac{1}{2} + \frac{1}{4} = \frac{7}{4} = \frac{14}{8}$



1.2. ábra. Nemnegatív gépi számok 4 bites tárolás esetében

Láthatjuk, hogy bármely tárolási módot használjuk, csak véges sok valós számot tudunk a számítógépen tárolni. Azokat a számokat, amelyeket pontosan, azaz tárolási hiba nélkül tudunk tárolni, *gépi számoknak* nevezzük. Azt a gépi számot, amelyet egy adott x valós szám helyett tárolunk a számítógépen, $\text{fl}(x)$ -szel jelöljük. Ha $|x|$ kisebb, mint a legkisebb ábrázolható pozitív szám, akkor definíció szerint $\text{fl}(x) = 0$, ha pedig $|x|$ nagyobb, mint a legnagyobb gépi szám, akkor legyen $\text{fl}(x) = \text{Inf}$. Az első esetben *alácsordulásról*, a másodikban *túlcsordulásról* beszélünk. Hogy definiálhatjuk $\text{fl}(x)$ -et a többi esetben? Két alapvető megközelítés lehetséges. Az egyik esetben vesszük az x szám bináris normál alakját, és annak mantisszájából annyi bitet tárolunk, amennyit az adott tárolási rendszerben tudunk, a többit elhagyjuk. Az egyszeres pontosságú számaábrázolás esetében tehát az első 23 törtbitet tároljuk. Ezt a stratégiát *levágásnak* nevezzük. A másik, gyakrabban használt megközelítés *kerekítést* használ. Ebben az esetben $\text{fl}(x)$ -et definiáljuk úgy, hogy legyen az x -hez legközelebbi gépi szám. Amikor x két egymás után következő gépi szám számtani közepe, akkor az előbbi definíció még nem határozza meg pontosan $\text{fl}(x)$ -et, mert ekkor kerekíthetünk felfelé és lefelé is. A már említett IEEE szabvány ebben az esetben is egyértelműen definiálja a kerekítést. A kerekítési szabályt az egyszeres pontosságú tárolásra fogalmazzuk meg. Vezessük be a következő jelöléseket: legyen az x pozitív valós szám bináris normál alakja $x = m2^k$, ahol $m = 1.m_1m_2 \dots m_{23}m_{24} \dots$. Legyen $x' = (1.m_1m_2 \dots m_{23})2^k$ és $x'' = ((1.m_1m_2 \dots m_{23})_2 + 2^{-23})2^k$. Ekkor x' és x'' egymás utáni gépi számok, és $x' \leq x \leq x''$, valamint $x'' - x' = 2^{k-23}$. A szabvány szerint legyen

$$\text{fl}(x) = \begin{cases} x', & \text{ha } |x - x'| < \frac{1}{2}|x'' - x'|, \\ x'', & \text{ha } |x - x''| < \frac{1}{2}|x'' - x'|, \\ x', & \text{ha } |x - x'| = \frac{1}{2}|x'' - x'| \text{ és } m_{23} = 0, \\ x'', & \text{ha } |x - x'| = \frac{1}{2}|x'' - x'| \text{ és } m_{23} = 1. \end{cases}$$

A határesetben, azaz ha $|x - x'| = \frac{1}{2}|x'' - x'|$ körülbelül az esetek felerészében fogunk így felfelé, és felerészben lefelé kerekíteni. A másik indoka ennek a definíciónak az, hogy ekkor a határesetben a kerekítéskor a mantissa utolsó bitje mindig 0 lesz, azaz a kerekített számon a 2-vel való osztás hiba nélkül végrehajtható. Kerekítést használva tehát az elkövetett hiba

$$|x - \text{fl}(x)| \leq \frac{1}{2}|x'' - x'| = \frac{1}{2}2^{-23}2^k.$$

Vizsgáljuk most a kerekítési hibát a pontos értékhez viszonyítva:

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{|x - \text{fl}(x)|}{(1.m_1m_2\dots)_2 \cdot 2^k} \leq \frac{1}{2}2^{-23}.$$

Könnyen látható, hogy az 1 gépi szám után következő első gépi szám $1 + 2^{-23}$ az előbb vizsgált 32 bites számábrázolási rendszerben. Ezt általánosítva jelölje $\varepsilon_{\text{gépi}}$ az adott számábrázolási rendszerben az első 1-nél nagyobb gépi szám és 1 különbségét. Ezt a számot *gépi epszilonnak* nevezzük. Eszerint $\varepsilon_{\text{gépi}}$ a legkisebb olyan 2 hatvány, amelyre a számítógépen az $1 + \varepsilon_{\text{gépi}} > 1$ egyenlőtlenség ellenőrizhető. Könnyen igazolható a következő tétel, amelyet az előbb a 32 bites bináris alapú tárolási rendszerre beláttunk:

1.9. tétel. Legyen $0 < \text{fl}(x) < \text{Inf}$, és tegyük fel, hogy a valós számokat kerekítve tároljuk. Ekkor

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{1}{2}\varepsilon_{\text{gépi}}.$$

A következő tétel bizonyítását az 5. feladatra hagyjuk.

1.10. tétel. Legyen b a valós számok ábrázolásakor használt számrendszer alapja, és t a mantissa tárolására használt bitek száma. Ekkor

$$\varepsilon_{\text{gépi}} = \begin{cases} 2^{-t}, & \text{ha } b = 2, \\ b^{1-t}, & \text{ha } b \neq 2. \end{cases}$$

Most definiáljuk a közelítés hibájának fogalmát, és egyéb, ehhez kapcsolódó fogalmakat. Legyen x egy valós szám, és tekintsük az $\tilde{x}$ valós számot x közelítésének. Ekkor a *közelítés hibáján* az $|x - \tilde{x}|$ számot értjük. Gyakran maga a hiba a számok nagyságrendjének ismerete nélkül nem mond túl sokat. Pl. az 10000 számnak az 10000.1 közelítését elég pontosnak érezzük, az 1-nek viszont az 1.1 nem túl jó közelítése, pedig mindkét esetben a közelítés hibája 0.1. Több információt jelent, ha a pontos értékhez viszonyítjuk a hibát. A *közelítés relatív hibáján* az

$$\frac{|x - \tilde{x}|}{|x|} \quad (x \neq 0)$$

számot értjük. Azt mondjuk, hogy a b alapú számrendszerben az $\tilde{x}$ közelítés n számjegyben pontos, ha

$$\frac{|x - \tilde{x}|}{|x|} \leq \frac{1}{2}b^{1-n}.$$

Látható, hogy minél kisebb a közelítés relatív hibája, annál nagyobb lesz a közelítésben szereplő pontos számjegyek száma. Ha $b = 10$, akkor úgy is megfogalmazhatjuk ezt a kapcsolatot a relatív hiba és a pontos számjegyek száma között, hogy egy nagyságrendi csökkenés (növekedés) a relatív hibában a pontos számjegyek számának egy jeggyel való növekedését (csökkenését) eredményezi.

1.11. példa. Legyen $x = 1657.3$ és $\tilde{x} = 1656.2$. Ekkor a közelítés hibája $|x - \tilde{x}| = 1.1$, relatív hibája $|x - \tilde{x}|/x = 0.0006637$. Mivel $|x - \tilde{x}|/x = 0.0006637 < 0.5 \cdot 10^{-2}$, ezért a közelítés az előbbi definíció értelmében 3 számjegyben pontos. Ha viszont x -et az $\tilde{x} = 1656.9$ számmal közelítjük, akkor ebben az esetben $|x - \tilde{x}|/x = 0.0002413 < 0.5 \cdot 10^{-3}$, azaz definíciónk szerint a közelítés 4 számjegyben pontos. $\square$

Az előbbi definíció és az 1.9. tétel szerint egyszeres pontosságú számábrázolás esetén az x valós szám helyett tárolt $\text{fl}(x)$ gépi szám 24 bináris számjegyben pontos. Minket általában a

tízes számrendszerben felírt alakban érdekel a pontos számjegyek száma. Az egyszeres pontosság esetében ezt megkapjuk, ha az

$$\frac{1}{2}2^{-23} \leq \frac{1}{2}10^{1-n}$$

egyenlőtlenséget teljesítő legnagyobb n egész számot megkeressük. Könnyen kiszámolható, hogy ez $n = 7$, azaz egyszeres számábrázolás esetében a tárolt gépi szám legalább 7 számjegyben pontos a tízes számrendszerben.

1.12. példa. Tekintsük az $x = 12.4$ valós számot. Írjuk fel először ennek bináris alakját. Könnyű ellenőrizni, hogy $12 = (1100)_2$. Keressük tehát a törtrésznek, 0.4 -nek a bináris alakját:

$$0.4 = (0.x_1x_2x_3\dots)_2 = \frac{x_1}{2} + \frac{x_2}{2^2} + \frac{x_3}{2^3} + \dots$$

Ha tehát 0.4 -nek vesszük a 2-szeresét, akkor annak egész része x_1 -et fogja megadni. $0.4 \cdot 2 = 0.8$, azaz $x_1 = 0$. Vesszük a szorzat törtrészét, 0.8 -at, és megismételjük az eljárást. $0.8 \cdot 2 = 1.6$, tehát $x_2 = 1$. A szorzat törtrésze 0.6 , amivel folytatjuk: $0.6 \cdot 2 = 1.2$, így $x_3 = 1$. Ennek a szorzatnak a törtrésze 0.2 . $0.2 \cdot 2 = 0.4$, ezért $x_4 = 0$, és 0.4 -gyel folytatjuk az eljárást. Látható, hogy ezután az eddigi jegyek, 0011 ismétlődnek ciklikusan végtelen sokszor, azaz $0.4 = (0.01100110011001100110011\dots)_2$. Az x szám bináris normál alakja tehát

$$x = 12.4 = (1.100011001100110011001100110011\dots)_2 \cdot 2^3.$$

x mantisszáját 23 bitre kerekítve (lefelé) kapjuk, hogy

$$\text{fl}(x) = (1.10001100110011001100110)_2 \cdot 2^3.$$

Ennek értéke a tízes számrendszerben felírva: $\text{fl}(x) = 12.3999996185302734375$. □

A számítógép által elvégzett gépi aritmetikai műveleteket a következőképpen lehet formálisan definiálni:

$$\begin{aligned} x \oplus y &:= \text{fl}(\text{fl}(x) + \text{fl}(y)), \\ x \ominus y &:= \text{fl}(\text{fl}(x) - \text{fl}(y)), \\ x \odot y &:= \text{fl}(\text{fl}(x) \cdot \text{fl}(y)), \\ x \oslash y &:= \text{fl}(\text{fl}(x) / \text{fl}(y)). \end{aligned}$$

Eszerint vesszük az adott műveletben szereplő tényezők gépi számra kerekített értékeit, azon elvégezzük a műveletet, majd a művelet eredményét kerekítjük a legközelebbi gépi számra.

Későbbi példáinkban gyakran fogunk hivatkozni az ún. *négyjegyű aritmetikára*. Ezen azt értjük, hogy olyan számábrázolási rendszert használunk, amely tízes alapú, és 4 mantissza jegyet tárol (és feltesszük, hogy elegendően sok helyünk van a számolás közben fellépő számok kitevőinek tárolásához). Ez azt jelenti, hogy minden egyes részletszámolás eredményét az első nem nulla számjegytől számított 4 jegyre, azaz az első 4 *értékes számjegyre* kerekítjük, és ezt használjuk tovább a számolás során. Négyjegyű aritmetikát használva a kerekítési hibák hatását fel tudjuk erősíteni a vizsgált példákban.

1.13. példa. Négyjegyű aritmetikát használva $1.043 + 32.25 = 33.29$, és hasonlóan $1.043 \cdot 32.25 = 33.64$ (kerekítés után). Viszont $1.043 + 20340 = 20340$ lesz, mivel négy értékes számjegyre kell kerekítenünk a 20341.043 pontos értéket. □

Feladatok

1. Váltsa át bináris alakra a következő tízes számrendszerben felírt számokat:

$$57, \quad -243, \quad 0.25, \quad 35.27$$

2. Írja fel a következő bináris számokat tízes alapú számrendszerben:

$$(101101)_2, \quad (0.10011)_2, \quad (1010.01101)_2$$

3. Mutassa meg, hogy a kettes komplement kódot negatív szám esetén megkaphatjuk a következőképpen: Vegyük a tárolandó negatív szám abszolút értékének bináris alakját. Cseréljük minden 0-t 1-re és 1-et 0-ra, majd adjunk 1-et a kapott számhoz.
4. Legyen I_1 és I_2 két m biten tárolt pozitív egész szám. Mutassa meg, hogy az $I_1 - I_2$ különbség kiszámítható úgy, hogy vesszük I_2 kettes komplement kódját, C_2 -t, és ehhez hozzáadjuk I_1 -et, majd vesszük az összeg utolsó m bitjét!
5. Bizonyítsa be az 1.10. tételt!
6. Írjon egy olyan programot, amely kiszámítja az adott számítógéphez és a használt számábrázolási rendszerhez tartozó gépi epsilon értékét!
7. Számítsa ki, hogy kétszeres pontosságú számábrázolás esetén hány számjegyben pontos a tárolt gépi szám!
8. Legyen $x = (x_0.x_1x_2 \dots x_mx_{m+1}x_{m+2} \dots) \cdot 10^k$, $\tilde{x} = (x_0.x_1x_2 \dots x_m\tilde{x}_{m+1}\tilde{x}_{m+2} \dots) \cdot 10^k$, azaz x és $\tilde{x}$ azonos nagyságrendűek, és az első $m+1$ db számjegyük megegyezik. Láss be, hogy ekkor $\tilde{x}$ legalább m számjegy pontosságú közelítése x -nek!

1.3. Hibaanalízis

Legyen x és y pozitív, és tekintsük az $\tilde{x}$ és $\tilde{y}$ számokat x és y közelítésének. Legyen $|x - \tilde{x}| \leq \Delta_x$ valamint $|y - \tilde{y}| \leq \Delta_y$ a közelítések hibakorlátja. A megfelelő relatív hibakorlátokat $\delta_x = \Delta_x/x$ és $\delta_y = \Delta_y/y$ jelölik. Ebben a szakaszban azzal a kérdéssel foglalkozunk, hogy ha az x és y számokon egy aritmetikai művelet (összeadás, kivonás, szorzás, osztás) kell elvégeznünk, és ahelyett az $\tilde{x}$ és $\tilde{y}$ számokon végezzük el a műveletet, és annak (pontos) eredményével közelítjük az eredeti művelet eredményét, mekkora lehet a közelítés hibája ill. relatív hibája.

Először tekintsük az összeadás műveletét. Keresünk tehát olyan Δ_{x+y} és δ_{x+y} számokat, hogy

$$|x + y - (\tilde{x} + \tilde{y})| \leq \Delta_{x+y} \quad \text{és} \quad \frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} \leq \delta_{x+y}.$$

1.14. tétel. A

$$\Delta_{x+y} := \Delta_x + \Delta_y \quad \text{és} \quad \delta_{x+y} := \max\{\delta_x, \delta_y\}$$

számok az összeadás hiba- ill. relatív hibakorlátai.

Bizonyítás. A háromszög-egyenlőtlenséget és Δ_x és Δ_y definícióját alkalmazva

$$|x + y - (\tilde{x} + \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y.$$

Ebből kapjuk hogy $\Delta_x + \Delta_y$ egy hibakorlátja lesz az összeadásnak.

Az előbbi összefüggést felhasználva

$$\frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} \leq \frac{\Delta_x + \Delta_y}{x + y} = \frac{x}{x + y} \delta_x + \frac{y}{x + y} \delta_y \leq \max\{\delta_x, \delta_y\}.$$

Tehát $\max\{\delta_x, \delta_y\}$ egy relatív hibakorlátja az összeadásnak. □

A tételt nyilván lehet általánosítani több szám összeadására: a tagok hibái összeadódhatnak, az összeg relatív hibája nem nagyobb, mint a tagok relatív hibái közül a legnagyobb. Az állítást megfogalmazhatjuk úgy is, hogy a közelítő összeg pontos jegyeinek száma nem kevesebb, mint az egyes tagok közelítéseiben szereplő pontos jegyek számai közül a legkisebb szám. Természetesen a tétel a legrosszabb esetre vonatkozik. A gyakorlatban a hibák kiegyenlíthetik egymást. Pl. legyen $x = 1$, $y = 2$, $\tilde{x} = 1.1$, $\tilde{y} = 1.8$. Ekkor $x + y = 3$, $\tilde{x} + \tilde{y} = 2.9$. Azaz az összeg hibája csak 0.1, kisebb, mint az egyes tagok hibáinak összege, 0.3.

1.15. tétel. Legyen $x > y > 0$. A

$$\Delta_{x-y} := \Delta_x + \Delta_y \quad \text{és} \quad \delta_{x-y} := \frac{x}{x-y}\delta_x + \frac{y}{x-y}\delta_y$$

számok a kivonás hiba- ill. relatív hibakorlátai.

Bizonyítás. Az

$$|x - y - (\tilde{x} - \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y$$

egyenlőtlenségekből következik az első állítás. Tekintsük az

$$\frac{|x - y - (\tilde{x} - \tilde{y})|}{x + y} \leq \frac{\Delta_x + \Delta_y}{x - y} = \frac{x}{x - y}\delta_x + \frac{y}{x - y}\delta_y,$$

becsléseket, amiből a második állítást kapjuk. $\square$

Látható, hogy ha egymáshoz közeli számokat vonunk ki egymásból, akkor a relatív hiba megsokszorozódhat, azaz a pontos számjegyek száma jelentősen csökkenhet. Ezt a jelenséget hívjuk *értékes számjegyek végzetes elvesztésének*.

1.16. példa. Legyen $x = 12.47531$, $\tilde{x} = 12.47534$, $y = 12.47326$, $\tilde{y} = 12.47325$, akkor $\delta_x = 2.4 \cdot 10^{-6}$ és $\delta_y = 8 \cdot 10^{-7}$. Viszont $x - y = 0.00205$, $\tilde{x} - \tilde{y} = 0.00209$, így $\delta_{x-y} = 0.0195$. Ellenőrizhetjük, hogy $\tilde{x}$ és $\tilde{y}$ 6 pontos számjegyet, $\tilde{x} - \tilde{y}$ viszont csak 2 pontos számjegyet tartalmaz. $\square$

1.17. tétel. Legyen $x, y > 0$. A

$$\Delta_{x \cdot y} := x\Delta_y + y\Delta_x + \Delta_x\Delta_y, \quad \text{és} \quad \delta_{x \cdot y} := \delta_x + \delta_y + \delta_x\delta_y$$

számok a szorzás hiba- ill. relatív hibakorlátai.

Bizonyítás. A háromszög-egyenlőtlenség szerint

$$\begin{aligned} |xy - \tilde{x}\tilde{y}| &= |xy - x\tilde{y} + x\tilde{y} - \tilde{x}\tilde{y}| \\ &\leq x|y - \tilde{y}| + |\tilde{y}||x - \tilde{x}| \\ &\leq x\Delta_y + |\tilde{y}|\Delta_x \\ &= x\Delta_y + |y + \tilde{y} - y|\Delta_x \\ &\leq x\Delta_y + y\Delta_x + \Delta_x\Delta_y. \end{aligned}$$

Az első állítás szerint a szorzat relatív hibája

$$\frac{|xy - \tilde{x}\tilde{y}|}{xy} \leq \frac{x\Delta_y + y\Delta_x + \Delta_x\Delta_y}{xy} = \delta_x + \delta_y + \delta_x\delta_y,$$

amiből kapjuk a második állítást. $\square$

Mivel Δ_x és Δ_y általában sokkal kisebb mint x és y , és így $\Delta_x \Delta_y$ elhanyagolható $x \Delta_y$ és $y \Delta_x$ -hez képest, ezért $x \Delta_y + y \Delta_x$ egy jó becslés a szorzat hibájára. Hasonlóan, $\delta_x + \delta_y$ jó közelítése a szorzat relatív hibakorlátjának.

1.18. tétel. Tegyük fel, hogy $x, y > 0$ és $\delta_y < 1$. Ekkor a

$$\Delta_{x/y} := \frac{x \Delta_y + y \Delta_x}{y(y - \Delta_y)} \quad \text{és} \quad \delta_{x/y} := \frac{\delta_x + \delta_y}{1 - \delta_y}$$

számok az osztás hiba- ill. relatív hibakorlátai.

Bizonyítás. Elemi átalakításokat használva kapjuk

$$\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right| = \frac{|x\tilde{y} - xy + xy - \tilde{x}y|}{y|\tilde{y}|} \leq \frac{x\Delta_y + y\Delta_x}{y|\tilde{y}|} = \frac{x\Delta_y + y\Delta_x}{y|y - (y - \tilde{y})|}.$$

A $\delta_y < 1$ feltételből következik, hogy $|y - \tilde{y}| \leq \Delta_y < y$, ezért az $|y - (y - \tilde{y})| \geq y - |y - \tilde{y}| \geq y - \Delta_y > 0$ egyenőtlenség felhasználásával következik a tétel első állítása.

A második állítás igazolásához tekintsük

$$\frac{\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right|}{\frac{x}{y}} = \frac{|x(\tilde{y} - y) - y(\tilde{x} - x)|}{x|\tilde{y}|} = \frac{\left| \frac{\tilde{y}-y}{y} - \frac{\tilde{x}-x}{x} \right|}{\left| 1 - \frac{y-\tilde{y}}{y} \right|} \leq \frac{\delta_x + \delta_y}{1 - \delta_y}.$$

□

Ha δ_y kicsi, akkor az osztás relatív hibakorlátját jól közelíti $\delta_x + \delta_y$. Hasonlóan, ha Δ_y y -hoz képest elhanyagolható, akkor $\frac{1}{y}\Delta_x + \frac{x}{y^2}\Delta_y$ jó becslése a $\Delta_{x/y}$ hibakorlátnak. Ha y sokkal kisebb, mint x , illetve ha y közel van 0-hoz, akkor Δ_y ill. Δ_x együtthatója nagy, azaz a hiba a tényezők hibáinak többszöröse lehet.

Feladatok

1. Legyen $x = 3.50$, $y = 10.00$, $\tilde{x} = 3.47$, $\tilde{y} = 10.02$. Adjön egy becslést a

$$3x + 7y, \quad \frac{1}{y}, \quad x^2, \quad y^3, \quad \frac{4xy}{x+y}$$

műveletek eredményének hibájára és relatív hibájára (a számítások elvégzése nélkül), ha azokban x és y helyett $\tilde{x}$ és $\tilde{y}$ -t használunk! Ezután számítsa ki a tényleges értékeket, hibákat és relatív hibákat, és hasonlítsa össze a kapott becslésekkel!

2. Legyen $\tilde{x}$ az x szám egy közelítése, és $|x - \tilde{x}| \leq \Delta_x$. Legyen $f: \mathbb{R} \rightarrow \mathbb{R}$ egy differenciálható függvény, amelyre $|f'(x)| \leq M$ minden $x \in \mathbb{R}$ -re. Legyen $y = f(x)$ és tekintsük az $\tilde{y} = f(\tilde{x})$ számot y közelítésének. Adjön becslést a közelítés hibájára! (Használja a Lagrange-féle középérték tételt!)

1.4. A véges számábrázolás következményei

1.19. példa. Oldjuk meg az

$$x^2 - 83.5x + 1.5 = 0$$

másodfokú egyenletet négyjegyű aritmetikát használva!

A másodfokú egyenlet megoldóképlete szerint és négyjegyű aritmetikát használva a numerikus megoldás

$$\tilde{x} = \frac{83.5 \pm \sqrt{83.5^2 - 4 \cdot 1.5}}{2} = \frac{83.5 \pm \sqrt{6972 - 6.000}}{2} = \frac{83.5 \pm 83.46}{2},$$

azaz

$$\tilde{x}_1 = \frac{167.0}{2} = 83.50, \quad \text{és} \quad \tilde{x}_2 = \frac{0.040}{2} = 0.020.$$

Az egyenlet pontos megoldása $x_1 = 83.482032$ ill. $x_2 = 0.0179679$. Ha kiszámoljuk a két gyök közelítésének relatív hibáit, a $\delta_1 = 0.0002152$ és $\delta_2 = 0.113096$ értékeket kapjuk. Az első numerikus gyök tehát 4 számjegy, a második viszont csak 1 számjegy pontosságú közelítés, azaz a két gyök pontossága között 3 nagyságrendi különbség van. Mi ennek az oka? A második gyök kiszámításakor a gyökképletben két egymáshoz közeli számot kellett kivonni egymásból. Az előző szakaszból tudjuk, hogy ez járhat a pontosság elvesztésével, és ezt tapasztalhattuk a jelenlegi számolásban is. $\square$

Tekintsük az $ax^2 + bx + c = 0$ másodfokú egyenlet két gyöke közül az

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \quad (1.2)$$

gyököt. Amikor b negatív, és $4ac$ sokkal kisebb, mint b^2 , két egymáshoz közeli számot vonunk ki egymásból a számlálóban, azaz fellép az értékes számjegyek végzetes elvesztésének jelensége. (Ezt az esetet vizsgáltuk az 1.19. példában.) Ennek kiküszöbölésére gyöktelenítsük a számlálót:

$$x_2 = \frac{b^2 - (b^2 - 4ac)}{2a(-b + \sqrt{b^2 - 4ac})} = \frac{2c}{-b + \sqrt{b^2 - 4ac}}. \quad (1.3)$$

Ez a képlet algebrailag ekvivalens az (1.2) formulával. A különbség viszont az, hogy ebben nem szerepel kivonás (a nevezőben két pozitív számot adunk össze). Ha b pozitív, akkor a másik gyökképlettel ismételhetjük meg ugyanezt a trükköt, és kaphatjuk az

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}} \quad (1.4)$$

formulát.

1.20. példa. Számítsuk ki az 1.19. példa második gyökét újra, négyjegyű aritmetikát és a gyökképlet (1.4) alakját használva!

$$\tilde{x}_2 = \frac{2 \cdot 1.5}{83.5 + \sqrt{83.5^2 - 4 \cdot 1.5}} = \frac{3}{83.5 + 83.46} = \frac{3}{167.0} = 0.01796.$$

Ennek a numerikus gyöknek a relatív hibája $\delta_2 = 0.00044$, azaz négy számjegy pontosságú a közelítés. $\square$

1.21. példa. Tegyük fel, hogy a $\cos^2 x - \sin^2 x$ kifejezést kell kiértékelnünk. Ha $x = \frac{\pi}{4}$, akkor a kifejezés pontos értéke 0, azaz ha x $\frac{\pi}{4}$ -hez közel van, akkor a kifejezés két egymáshoz közeli szám különbsége lesz, ahol fellép a pontosság elvesztése. Ezt könnyen kikerülhetjük, ha az eredeti kifejezés helyett az azzal algebrailag ekvivalens $\cos 2x$ alakot használjuk. $\square$

Az eddigi példáinkban algebrai azonosságot használva tudtuk a pontosság elvesztését megakadályozni. A következő példákban ugyanezt más módszerrel tesszük meg.

1.22. példa. Tekintsük az $f(x) = e^x - 1$ függvényt. Az $x = 0$ közelében ismét két közel azonos számot kell egymásból kivonni, viszont most nincs olyan azonosság, amellyel ezt el lehetne kerülni. Ha e^x Taylor-sorát vesszük, akkor az 1-gyel való kivonással tudunk egyszerűsíteni:

$$f(x) = x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots$$

Tehát f -et érdemes ennek a végtelen sornak egy véges közelítő összege segítségével kiértékelni. $\square$

Egy más jellegű problémát vet fel a következő példa.

1.23. példa. Számítsuk ki az $y = 20^{50}/50!$ szám értékét! A probléma a következő: ha a képlet alapján először a számlálót és a nevezőt külön akarjuk kiszámolni, rögtön beleütközünk a számábrázolás szabta korlátokba, egyszeres pontosság használata esetén már túlsordul a számolás. Másrészt tudjuk, hogy $a^n/n! \rightarrow 0$, ha $n \rightarrow \infty$, így a számolás végeredménye várhatóan kis szám lesz. Rendezzük úgy a számítást, hogy minden részeredmény benne maradjon az ábrázolható számok tartományában:

$$\frac{20^{50}}{50!} = \frac{20}{50} \cdot \frac{20}{49} \cdot \frac{20}{48} \cdots \frac{20}{1}.$$

Ezt a képletet a számítógépen egy egyszerű **for** ciklussal kiszámolhatjuk:

```
y ← 20
for i = 2, ..., 50 do
    y ← y ·  $\frac{20}{i}$ 
end do
output(y)
```

A számolás eredménye: 3.701902 (6 tizedesjegy pontossággal). □

1.24. példa. Számítsuk ki az

$$A = 10.00 + 0.002 + 0.002 + \cdots + 0.002 = 10.00 + \sum_{i=1}^{10} 0.002$$

összeget, négyjegyű aritmetikát használva! Balról jobbra értékeljük ki az összeadásokat, így először az $10.00 + 0.002$ összeget kell kiszámítanunk. Négyjegyű aritmetika szerint $10.00 + 0.002 = 10.002 = 10.00$ kerekítés után. Ehhez hozzáadva a következő számot, a 4 jegyre kerekítés miatt, újra $10.00 + 0.002 + 0.002 = 10.00$ lesz. Látható, hogy $A = 10.00$ lesz a számolás eredménye.

Nézzük most újra az előbbi összeadást, de más sorrendben:

$$B = 0.002 + 0.002 + \cdots + 0.002 + 10.00 = \sum_{i=1}^{10} 0.002 + 10.00.$$

Most először a $0.002 + 0.002 = 0.004$ összeget számítjuk ki. Ezt a négyjegyű aritmetikában is pontosan tudjuk számolni! Ezután sorra számolható: $0.002 + 0.002 + 0.002 = 0.006$ stb, végül $\sum_{i=1}^{10} 0.002 = 0.02$. Az eredmény tehát ebben a sorrendben $B = 10.02$. Ezen összeadások egyikében sem lépett fel kerekítési hiba, mivel minden részeredményt pontosan tárolhattunk a négyjegyű aritmetikában.

Ez a példánk mutatja azt is, hogy a kettőnél több tagú összeadás nem kommutatív művelet a számítógépeken. □

Az előző példa tanulsága az, hogy, amikor lehet, az összeadásokat érdemes a tagok növekvő sorrendjében végezni, mert ekkor van a legnagyobb esély arra, hogy a számolás során kapott részösszeg azonos nagyságrendű legyen a következő összeadandóval, és így minél kisebb legyen a kerekítési hiba.

Feladatok

- Vizsgálja meg, hogy a következő kifejezésekben mikor lép fel az értékes számjegyek elvesztésének jelensége! Hogyan tudjuk kiküszöbölni a pontosság csökkenését?

- $\ln x - 1$,
- $\sqrt{x+9} - 3$,
- $\sin x - x$,

- (d) $1 - \cos x$,
 - (e) $(1 - \cos x)/\sin x$,
 - (f) $(\cos x - e^{-x})/x$,
2. Négyjegyű aritmetikát használva számítsa ki az $2.274 + 12.04 + 0.4233 + 0.1202 + 0.2204$ összeget, majd rendezze a tagokat növekvő sorrendbe, és úgy is számítsa ki az összeget!